

Playwright Stabilization Demo Report

1.Objective

The objective of this project is to demonstrate advanced software testing techniques using Playwright

- Flaky test identification and stabilization
 - Diagnostics and structured logging
 - Offline resilience testing
 - Idempotency verification
 - Evidence collection
-

2.Environment

Framework – Playwright

Logging – Winston

3.Framework Design

The framework follows a layered architecture.

Configuration Layer

Centralizes reusable configuration.

Files:

playwright.config.ts

Fixtures Layer

Provides reusable fixtures.

Files:

evidence-fixture.ts

logging-fixture.ts

Test Layer

Contains UI and API automation.

Files:

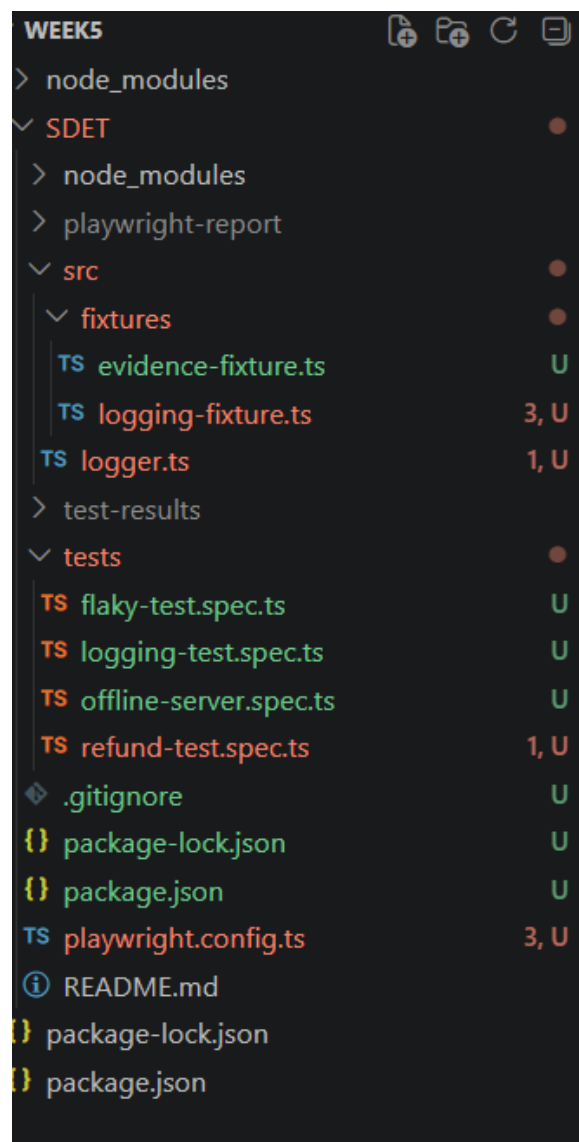
offline-server.spec.ts

flaky-test.spec.ts

logging-test.spec.ts

refund-test.spec.ts

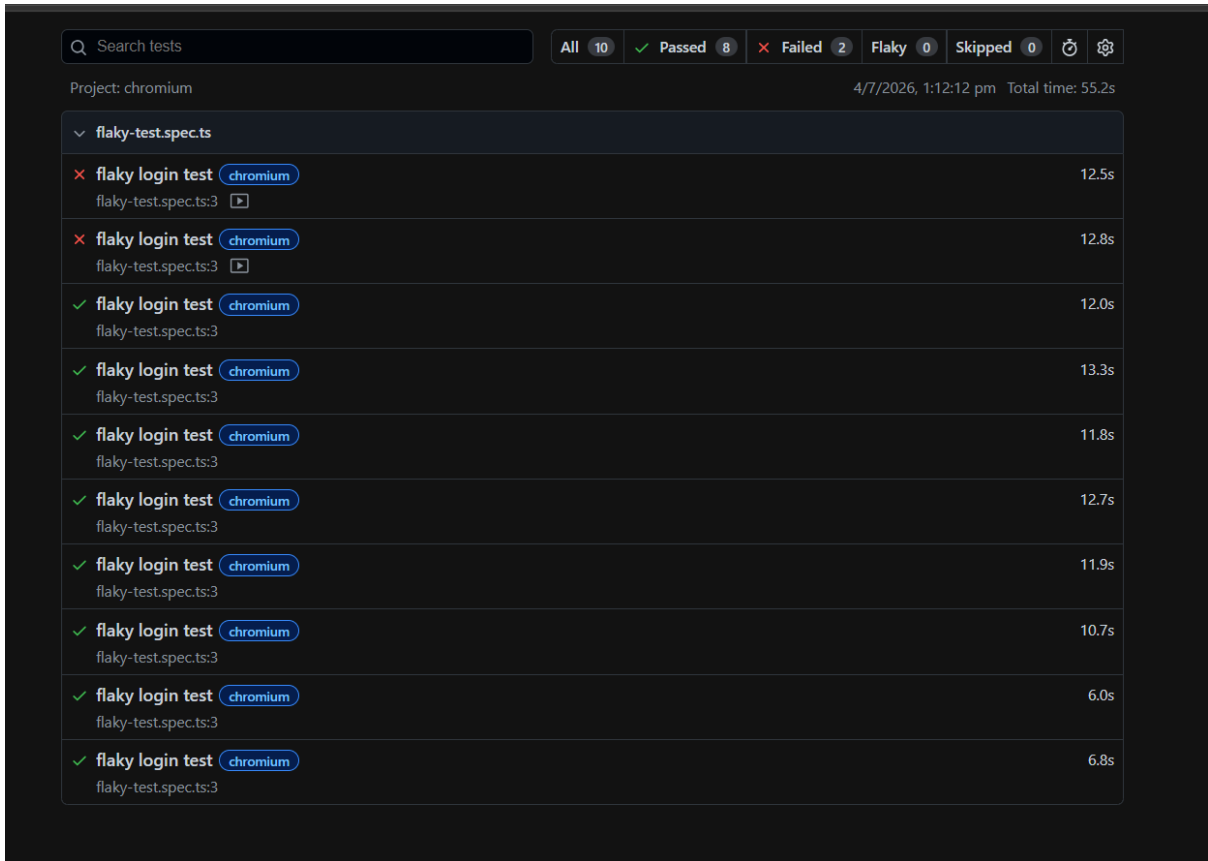
4. Project Structure



5. Flaky Test Analysis

To intentionally introduce synchronization issues in UI tests, observe failures, identify the root cause, and stabilize the tests using Playwright best practices.

Scenario 1 – Login Navigation



The screenshot displays the Playwright test runner interface. At the top, there is a search bar and a summary bar showing 'All 10', 'Passed 8', 'Failed 2', 'Flaky 0', and 'Skipped 0'. Below this, the project name 'chromium' and the date/time '4/7/2026, 1:12:12 pm' are shown, along with the total time 'Total time: 55.2s'. The main table lists 10 test runs for 'flaky-test.spec.ts'. The first two runs are marked as failed (red X), and the remaining eight are marked as passed (green checkmark). Each row includes a 'chromium' label and a duration in seconds.

Test Name	Browser	Duration	Status
flaky login test	chromium	12.5s	Failed
flaky login test	chromium	12.8s	Failed
flaky login test	chromium	12.0s	Passed
flaky login test	chromium	13.3s	Passed
flaky login test	chromium	11.8s	Passed
flaky login test	chromium	12.7s	Passed
flaky login test	chromium	11.9s	Passed
flaky login test	chromium	10.7s	Passed
flaky login test	chromium	6.0s	Passed
flaky login test	chromium	6.8s	Passed

The assertion timeout was intentionally reduced to **50 ms**, while the application requires varying amounts of time to authenticate and navigate to the Home page.

Stabilization

Removed the artificial timeout and relied on Playwright's automatic waiting.

Scenario 2 – Apply coupon

Q Search tests	All 10	✓ Passed 2	✗ Failed 8	Flaky 0	Skipped 0	🔄	⚙️
Project: chromium		4/7/2026, 2:57:26 pm Total time: 56.2s					
▼ flaky-test.spec.ts							
✗ Apply coupon - flaky	chromium						12.9s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						15.5s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						14.7s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						13.2s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						16.8s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						16.9s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						10.2s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✗ Apply coupon - flaky	chromium						6.4s
flaky-test.spec.ts:29 ▶️ 📄 View Trace							
✓ Apply coupon - flaky	chromium						14.3s
flaky-test.spec.ts:29 📄 View Trace							
✓ Apply coupon - flaky	chromium						6.7s
flaky-test.spec.ts:29 📄 View Trace							

Flakiness is introduced by not waiting for the “place order” to be visible.

Stabilization

Removed the artificial timeout and relied on Playwright's automatic waiting.

6.Logging test analysis

Scenario – Checkout journey with logs

A unique Correlation ID is generated for every test execution.

Sensitive information is automatically masked before being written to logs.

Protected fields include authorization, password, card number, CVV , token

```
C:\Users\299164\Downloads\playwright repos\week5\SDET>npx playwright test tests/logging-test.spec.ts --headed

Running 1 test using 1 worker
[chromium] > tests\logging-test.spec.ts:5:5 > checkout journey with logs
[INFO] 2026-07-04T11:40:50.988Z - test started
[INFO] 2026-07-04T11:40:50.991Z - checkout journey starts
[INFO] 2026-07-04T11:40:52.184Z - login page opened
[INFO] 2026-07-04T11:40:52.296Z - credentials entered
[INFO] 2026-07-04T11:40:53.468Z - login successful
[INFO] 2026-07-04T11:40:53.818Z - catalog opened
[INFO] 2026-07-04T11:40:54.145Z - product opened {"product":"Running Shoes"}
[INFO] 2026-07-04T11:40:54.242Z - item added {"productId":101,"quantity":1}
[INFO] 2026-07-04T11:40:54.485Z - checkout opened
[INFO] 2026-07-04T11:40:54.530Z - coupon entered {"coupon":"UST10"}
[INFO] 2026-07-04T11:40:54.552Z - discount applied {"discount":450}
[INFO] 2026-07-04T11:40:55.033Z - order placed successfully
[INFO] 2026-07-04T11:40:55.039Z - test finished {"status":"passed"}
1 passed (13.1s)
```

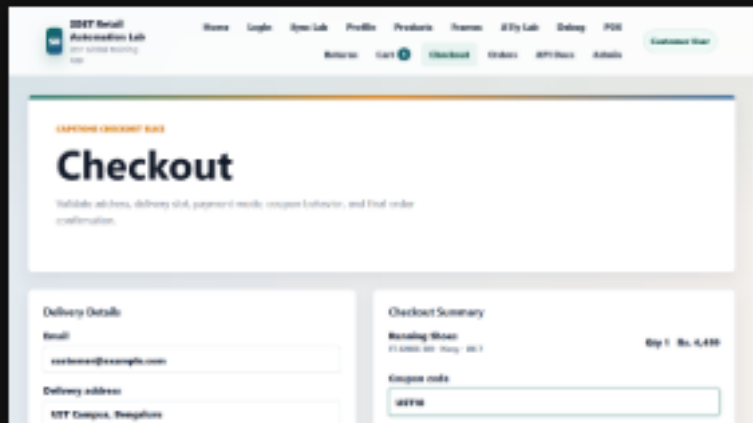
7.Evidence Collection analysis

To automatically capture execution artifacts during test execution to simplify debugging, root cause analysis, and failure investigation.

Evidence captured as:

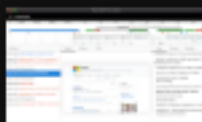
- Screenshot
- Video recording
- Playwright trace
- HTML report
- Structured Logs as JSON

Screenshots



 [screenshot](#)

Trace



 [trace](#)

Video



 [video](#)

Attachments

 [login.json](#)

 [error-console](#)

>  [stdout](#)

> Executed in Worker #0

8.Offline Resilience Testing

Verify the application's behavior under various network conditions, including offline mode and synchronization after network recovery.

Scenario 1 – Offline banner & Queue synchronization

Scenario 2 – Order confirmation request failure

Scenario 3 – Rollback on failed sale synchronization

Scenario 4 – Synchronization after flaky reconnect

Q Search tests

All4

✓ Passed4

Failed0

Flaky0

Skipped0

Project: chromium

4/7/2026, 5:22:45 pm Total time: 36.1s

▼ offline-server.spec.ts

✓ offline bannerchromium20.5s

offline-server.spec.ts:4 View Trace

✓ order confirmation request failschromium22.1s

offline-server.spec.ts:25 View Trace

✓ Rollbackchromium18.0s

offline-server.spec.ts:50 View Trace

✓ sync once, even through a flaky reconnectchromium21.3s

offline-server.spec.ts:62 View Trace

9.Refund testing

Scenario 1 – Full Refund

Scenario 2 – Partial Refund

Scenario 3 – Multiple Partial Refunds

Scenario 4 – Idempotent Partial Refund

Search tests		All 4	✓ Passed 4	Failed 0	Flaky 0	Skipped 0		
Project: chromium		4/7/2026, 5:26:41 pm Total time: 2.3s						
▼ refund-test.spec.ts								
✓	Full refund	chromium	324ms					
	refund-test.spec.ts:86			View Trace				
✓	Partial refund -1 of 4 shoes	chromium	256ms					
	refund-test.spec.ts:130			View Trace				
✓	Multiple partial refunds on same order	chromium	380ms					
	refund-test.spec.ts:182			View Trace				
✓	Partial refund idempotent	chromium	278ms					
	refund-test.spec.ts:279			View Trace				

10.Conclusion

The Playwright automation framework successfully validated UI workflows, refund APIs, offline resilience, and diagnostics. Reusable fixtures, structured logging, evidence collection, and flaky test stabilization improved the reliability and maintainability of the test suite.